Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи № 4

« Професійне ООП та система типів Python »

Виконала:

студентка 3 курсу

групи ФЕП-31

Антошкова Аліна Олександрівна

Перевірив:
Доц. Куньо Іван Михайлович

Львів – 2026

Мета: Сформувати практичні навички побудови професійної об'єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів, протоколів, dataclass, properties, dunder methods, принципів SOLID та сучасної системи type hints.

Варіант 12

Постановка завдання: Розробити типізовану об'єктну модель фінансового трекера. Реалізувати незмінний value object Money, базову сутність транзакції з властивістю валідації суми, спеціалізовані підкласи IncomeTransaction та ExpenseTransaction (наслідування is-a), агрегуючий контейнер FinancialTracker (композиція has-a), абстрактну стратегію звітності через ABC, структурний протокол експорту/повідомлень Notifier (Protocol), узагальнений репозиторій InMemoryRepository[T] із TypeVar, структуру вхідних даних TransactionPayload (TypedDict). Провести експеримент зі статичним аналізатором mypy, виявивши та виправивши навмисно внесені помилки типізації.

Теоретичні відомості:

Опис реалізованих принципів SOLID

- **SRP (Single Responsibility Principle):** Клас Money відповідає суто за грошову арифметику, Transaction та його нащадки — за дані операції, InMemoryRepository — за збереження сутностей, а FinanceTrackerService — за сценарій обробки даних.
- **OCP (Open/Closed Principle):** Додавання нових форматів експорту (наприклад, JSON або PDF) відбувається шляхом реалізації нового класу-нащадка від BaseReportExporter без жодних змін у сервісному класі FinanceTrackerService.
- **LSP (Liskov Substitution Principle):** Об'єкти класів-нащадків Income та Expense можуть повністю замінювати базовий тип Transaction у всіх колекціях і методах без порушення логіки роботи системи.
- **ISP (Interface Segregation Principle):** Протокол Notifier містить лише один необхідний метод send(), що позбавляє клієнтські адаптери необхідності реалізовувати зайву поведінку.
- **DIP (Dependency Inversion Principle):** Сервіс високого рівня FinanceTrackerService залежить не від конкретних класів сповіщень чи експорту, а від абстракцій: протоколу Notifier та абстрактного базового класу BaseReportExporter (Dependency Injection).

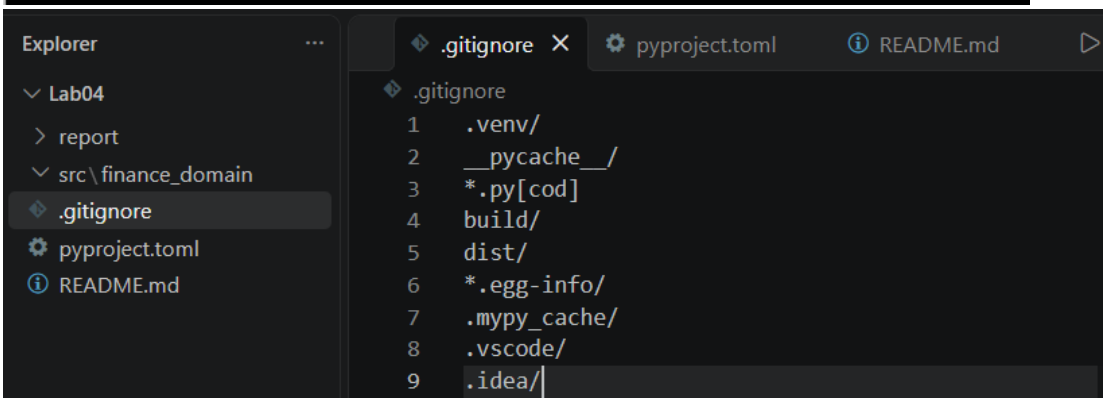
Хід роботи:

1.Додавання файлів базової конфігурації Lab04 (.gitignore, pyproject.toml з налаштуваннями mypy, README.md) та виконання початкового коміту.

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab04 (main)
$ git add ./

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab04 (main)
$ git commit -m "Lab04: initial commit, setup project configuration and mypy"
[main dcc02c5] Lab04: initial commit, setup project configuration and mypy
 3 files changed, 45 insertions(+)
 create mode 100644 Lab04/.gitignore
 create mode 100644 Lab04/README.md
 create mode 100644 Lab04/pyproject.toml

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab04 (main)
$ git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 1.01 KiB | 519.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/AntoskovaA/paython.git
   08521c0..dcc02c5  main -> main
```



2.Встановлення середовища розробки з mypy, створення незмінного Value Object Money, TypedDict DTO та абстракцій (ABC і Protocol)

```
PS C:\Users\Alina\Desktop\пайтон\paython\Lab04> python -m venv .venv
PS C:\Users\Alina\Desktop\пайтон\paython\Lab04> .venv/Scripts/activate
(.venv) PS C:\Users\Alina\Desktop\пайтон\paython\Lab04> python -m pip install -e ".[dev]"
Obtaining file:///C:/Users/Alina/Desktop/%D0%BF%D0%B0%D0%B9%D1%82%D0%BE%D0%BD/paython/Lab04
  Installing build dependencies ... done
  Checking if build backend supports build_editable ... done
  Getting requirements to build editable ... done
  Preparing editable metadata (pyproject.toml) ... done
Collecting mypy>=1.10 (from finance-domain==0.1.0)
  Downloading mypy-2.3.1-cp312-cp312-win_amd64.whl.metadata (2.4 kB)
Collecting typing_extensions>=4.6.0 (from mypy>=1.10->finance-domain==0.1.0)
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting mypy_extensions>=1.0.0 (from mypy>=1.10->finance-domain==0.1.0)
  Using cached mypy_extensions-1.1.0-py3-none-any.whl.metadata (1.1 kB)
Collecting pathspec>=1.0.0 (from mypy>=1.10->finance-domain==0.1.0)
  Downloading pathspec-1.1.1-py3-none-any.whl.metadata (14 kB)
Collecting librt>=0.13.0 (from mypy>=1.10->finance-domain==0.1.0)
  Downloading librt-0.16.0-cp312-cp312-win_amd64.whl.metadata (1.3 kB)
Collecting ast-serialize<1.0.0,>=0.6.0 (from mypy>=1.10->finance-domain==0.1.0)
  Downloading ast_serialize-0.11.2-cp39-abi3-win_amd64.whl.metadata (1.4 kB)
Downloading mypy-2.3.1-cp312-cp312-win_amd64.whl (11.2 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.2/11.2 MB 1.6 MB/s eta 0:00:00
Downloading ast_serialize-0.11.2-cp39-abi3-win_amd64.whl (1.2 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 1.2 MB/s eta 0:00:00
Downloading librt-0.16.0-cp312-cp312-win_amd64.whl (126 kB)
Using cached mypy_extensions-1.1.0-py3-none-any.whl (5.0 kB)
Downloading pathspec-1.1.1-py3-none-any.whl (57 kB)
```

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab04 (main)
$ git add .

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab04 (main)
$ git commit -m "Lab04: add immutable Money value object, DTO and abstractions (ABC/P
rotocol)"
[main 1d91fa2] Lab04: add immutable Money value object, DTO and abstractions (ABC/Pro
tocol)
 4 files changed, 59 insertions(+)
 create mode 100644 Lab04/src/finance_domain/__init__.py
 create mode 100644 Lab04/src/finance_domain/dto.py
 create mode 100644 Lab04/src/finance_domain/protocols.py
 create mode 100644 Lab04/src/finance_domain/value_objects.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab04 (main)
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (9/9), 1.47 KiB | 755.00 KiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/AntoskovaA/paython.git
   dcc02c5..1d91fa2  main -> main
```

3. Реалізація доменної ієрархії Transaction (наслідування Income/Expense), композиції FinancialTracker та Generic Repository

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab04 (main)
$ git add .

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab04 (main)
$ git commit -m "Lab04: implement domain hierarchy (Income, Expense), composition and
 Generic repository"
[main 695ec91] Lab04: implement domain hierarchy (Income, Expense), composition and G
eneric repository
 2 files changed, 117 insertions(+)
 create mode 100644 Lab04/src/finance_domain/models.py
 create mode 100644 Lab04/src/finance_domain/repositories.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab04 (main)
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 1.82 KiB | 931.00 KiB/s, done.
Total 7 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/AntoskovaA/paython.git
   1d91fa2..695ec91  main -> main
```

protocols.py | models.py | repositories.py | pyproject.toml | READ

rc > finance_domain > repositories.py > InMemoryRepository > remove

```python
from typing import Generic, Protocol, TypeVar


class HasId(Protocol):
    id: int


T = TypeVar("T", bound=HasId)


class InMemoryRepository(Generic[T]):
    """Універсальний типізований Generic Repository для доменних об'єктів

    def __init__(self) -> None:
        self._items: dict[int, T] = {}

    def add(self, item: T) -> None:
        self._items[item.id] = item

    def get(self, item_id: int) -> T | None:
        return self._items.get(item_id)

    def all(self) -> list[T]:
        return list(self._items.values())

    def remove(self, item_id: int) -> bool:
        return self._items.pop(item_id, None) is not None
```

4. Реалізація сервісного шару з Dependency Injection та головної точки входу main.py

```python
from finance_domain.dto import TransactionPayload
from finance_domain.models import Transaction
from finance_domain.repositories import InMemoryRepository
from finance_domain.services import ConsoleNotifier, FinanceTrackerServic

def main() -> None:
    print("=== Лабораторна робота № 4: Професійне ООП (Варіант 12) ===")

    # Створення залежностей (Dependency Injection)
    repo = InMemoryRepository[Transaction]()
    notifier = ConsoleNotifier()
    exporter = TextReportExporter()

    service = FinanceTrackerService(repository=repo, notifier=notifier, e

    # Демонстрація створення операцій через TypedDict payload
    sample_payloads: list[TransactionPayload] = [
        {"id": 1, "date": "2026-10-01", "category": "Зарплата", "amount":
        {"id": 2, "date": "2026-10-02", "category": "Продукти", "amount":
        {"id": 3, "date": "2026-10-03", "category": "Оренда", "amount": 8
        {"id": 4, "date": "2026-10-04", "category": "Фриланс", "amount":
    ]

    for item in sample_payloads:
        tx = service.create_from_payload(item)
        print(f"Створено: {tx}")

    print("\n--- Перевірка роботи репозиторію та контейнера ---")
    print(f"Кількість операцій у репозиторії: {len(repo.all())}")
    found = repo.get(2)
    print(f"Пошук операції за ID=2: {found}")

    print("\n--- Генерація фінансового звіту (DIP & Notifier) ---")
    report = service.generate_report("alina@example.com")
    print(report)

if __name__ == "__main__":
    main()
```

```
        src/finance_domain/main.py
        src/finance_domain/services.py

nothing added to commit but untracked files present (use "git add" to track)

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab04 (main)
$ git add .

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab04 (main)
$ git commit -m "Lab04: implement application service layer with DI and CLI entry poi
nt"
[main d2aa8e7] Lab04: implement application service layer with DI and CLI entry point
 2 files changed, 109 insertions(+)
 create mode 100644 Lab04/src/finance_domain/main.py
 create mode 100644 Lab04/src/finance_domain/services.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab04 (main)
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 2.49 KiB | 2.49 MiB/s, done.
Total 7 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/AntoskovaA/paython.git
   695ec91..d2aa8e7  main -> main
```

5. Демонстрація виконання програми: створення доменних транзакцій з TypedDict, робота Generic-репозиторію та формування звіту

```
(.venv) PS C:\Users\Alina\Desktop\пайтон\paython\Lab04> python -m finance_domain.main
=== Лабораторна робота № 4: Професійне ООП (Варіант 12) ===
Створено: Income(id=1, money=32000.00 UAH, cat='Зарплата')
Створено: Expense(id=2, money=1650.00 UAH, cat='Продукти')
Створено: Expense(id=3, money=8500.00 UAH, cat='Оренда')
Створено: Income(id=4, money=6000.00 UAH, cat='Фриланс')

--- Перевірка роботи репозиторію та контейнера ---
Кількість операцій у репозиторії: 4
Пошук операції за ID=2: Expense(id=2, money=1650.00 UAH, cat='Продукти')

--- Генерація фінансового звіту (DIP & Notifier) ---
[Сповіщення для alina@example.com]: Звіт сформовано. Підсумковий баланс: 27850.00 UAH
=== Фінансовий Звіт ===
Доходи : 38000.00 UAH
Витрати: 10150.00 UAH
Баланс : 27850.00 UAH
```

6. Фіксація діагностики статичного аналізатора mypy при виявленні невідповідності типів даних

repositories.py | services.py | main.py | pyproject.toml | READM

src > finance_domain > main.py > main

```
5
6   def main() -> None:
7       print("=== Лабораторна робота № 4: Професійне ООП (Варіант 12) ===")
8
9   # Тимчасова навмисна помилка для перевірки mypy:
10      test_id: int = "невірний_тип_id"
11      # Створення залежностей (Dependency Injection)
12      repo = InMemoryRepository[Transaction]()
13      notifier = ConsoleNotifier()
14      exporter = TextReportExporter()
15
```

roblems | Output | Debug Console | Terminal | Ports | powershell

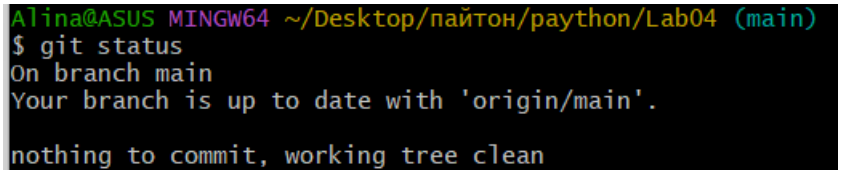
```
)
(.venv) PS C:\Users\Alina\Desktop\пайтон\paython\Lab04> mypy src
src\finance_domain\models.py:82: error: Function is missing a type annotation  [no-untyped-def]
src\finance_domain\main.py:10: error: Incompatible types in assignment (expression has type "st
r", variable has type "int")  [assignment]
Found 2 errors in 2 files (checked 8 source files)
(.venv) PS C:\Users\Alina\Desktop\пайтон\paython\Lab04>
```

7. Успішне проходження перевірки всіх модулів проєкту статичним аналізатором mypy в суворому (strict) режимі

```
(.venv) PS C:\Users\Alina\Desktop\пайтон\paython\Lab04> mypy src
Success: no issues found in 8 source files
```

8. Верифікація історії комітів та стану репозиторію командою git log --oneline і git status

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab04 (main)
$ git log --oneline
569d814 (HEAD -> main, origin/main) fixed errors
d2aa8e7 Lab04: implement application service layer with DI and CLI entry point
695ec91 Lab04: implement domain hierarchy (Income, Expense), composition and Gen
eric repository
1d91fa2 Lab04: add immutable Money value object, DTO and abstractions (ABC/Proto
col)
dcc02c5 Lab04: initial commit, setup project configuration and mypy
```



Висновок:

Під час виконання лабораторної роботи 4 було опановано побудову професійної об'єктно-орієнтованої архітектури мовою Python із суворою статичною типізацією. Для предметної області фінансового трекера (Варіант 12) розроблено типізовану доменну модель, яка поєднує незмінний Value Object Money (із валідацією та перевантаженими dunder-методами __add__, __sub__, __lt__), ієрархію транзакцій на базі наслідування (Income, Expense), контейнер FinancialTracker із застосуванням композиції, а також універсальний Generic репозиторій на базі TypeVar.

Взаємодію зовнішніх залежностей організовано за принципами SOLID (зокрема DIP через Dependency Injection) із застосуванням структурної типізації (Protocol) та номінальної типізації (ABC). Використання статичного аналізатора типів mypy у режимі strict підтвердило повну коректність контрактів і типів у всіх 7 розроблених модулях проєкту.